SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 4 **Total: Marks:40**

Annexure A

Code of Conduct:

I Priyanka.G/192524124 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *Priyanka.G*

Student 31: Priyanka G | Register Number: 192524124

INDUSTRY PROBLEM TASK

INDUSTRY SCENARIO

Context: An e-commerce platform has collected user records (browsing history, purchase history, category preferences, average spend, cart abandonment rate) to build a predictive model for identifying likely purchase conversion. The platform also wants to train an AI agent to recommend pricing/marketing actions (Discount / Bundle Offer / Highlight Product / No Action) based on a user's shopping session behaviour.

You are hired as an AI/ML Engineer. Address the following tasks:

Task 1: Data Preparation & Inductive Learning

(a) Describe the inductive learning approach suitable for this e-commerce dataset. Identify the target variable and at least three key features with justification.

(b) Explain how you would handle class imbalance (far fewer converters than non-converters) in the dataset. Suggest and justify a suitable balancing strategy (SMOTE / under-sampling / class weights).

(c) Split the dataset (70:30) and justify the split ratio for a conversion-prediction use-case. State the preprocessing steps (normalisation, encoding) applied and their impact.

Task 2: Decision Tree Modeling

(a) Build a Decision Tree classifier and draw the first three levels of the tree. Justify the root node selection using Information Gain / Gini Index values.

(b) Evaluate the model: report Accuracy, Precision, Recall, and F1-Score. Identify False Negatives (missed likely converters) and suggest how to reduce them - justify from a revenue perspective.

(c) Apply post-pruning (cost-complexity pruning). Compare pre-pruning vs post-pruning accuracy and explain which model is more appropriate for deployment in a live recommendation engine.

Task 3: Statistical Learning for Risk Stratification

(a) Apply Naive Bayes or Logistic Regression as a statistical learning model on the same dataset. Compare its performance (Accuracy, AUC-ROC) with the Decision Tree. Discuss when a statistical model is preferable.

(b) Perform feature importance analysis. Identify the top 3 predictors of conversion from both the Decision Tree and the statistical model. Do they agree? Justify your findings.

Task 4: Reinforcement Learning for Action Recommendation

(a) Model the pricing/marketing recommendation as an MDP. Define: States (user session stages), Actions (Discount / Bundle Offer / Highlight Product / No Action), Reward function (conversion/revenue improvement score), and Transition dynamics. Justify each component.

(b) Apply Q-Learning to train the agent for at least 5 user-session episodes. Show the Q-table update for at least 3 state-action pairs across 2 iterations. State the convergence criterion used.

(c) Extract the final learned policy. Discuss ethical considerations of deploying a Reinforcement Learning agent for dynamic pricing (price discrimination, manipulation of vulnerable users, transparency).

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context	
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts	
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution	
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools	
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis	
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation	

INDUSTRY SCENARIO

An e-commerce platform records browsing history, purchase history, category preferences, average spend and cart-abandonment rate for each user session. Two AI systems are required: (i) a supervised classification model that predicts purchase conversion, and (ii) a reinforcement-learning agent that recommends a pricing/marketing action - Discount, Bundle Offer, Highlight Product, or No Action - for each session.

TASK 1: DATA PREPARATION & INDUCTIVE LEARNING

(a) Inductive Learning Approach, Target Variable and Key Features

Inductive learning is the appropriate paradigm here because the platform already holds a large volume of labelled historical sessions (session ended in a purchase / did not) and the goal is to generalise a hypothesis from these specific examples to predict conversion for future, unseen sessions. This is supervised, example-driven learning rather than knowledge-based (deductive) reasoning, since no formal domain rules for "will this user buy" exist a priori - the model must induce the pattern purely from data. A version-space / decision-tree style inductive learner is well suited because e-commerce behavioural data mixes numeric and categorical attributes and the business also wants an interpretable model that marketing staff can read.

Target variable: Conversion_Flag - a binary label (1 = user completed a purchase in the session, 0 = user did not) derived from the transaction log.

Key features (minimum three, with justification):

- Cart Abandonment Rate - a session with items added but not checked out is the single strongest historical signal of purchase intent that stalled; it directly separates near-converters from browsers.
- Average Spend (historical) - captures the user's economic behaviour and price sensitivity; high past spend correlates with repeat conversion and helps the model distinguish loyal buyers from window-shoppers.
- Category Preference Match - whether the products viewed in the current session align with the user's historically preferred category; a strong match indicates focused intent rather than random browsing.
- Browsing History Depth (pages / products viewed) - session engagement depth is a proxy for consideration stage and adds discriminative power alongside the three features above.

Justification: These four features were chosen because they span behavioural (cart abandonment, browsing depth), transactional (average spend) and preference-based (category match) signal types, giving the inductive learner complementary evidence rather than redundant correlated inputs, which improves generalisation and interpretability for the business stakeholders.

(b) Handling Class Imbalance

In conversion-prediction datasets, converters are typically 2%-10% of sessions, so a model trained naively will over-predict the majority "no-conversion" class and miss most real buyers. The recommended strategy is SMOTE (Synthetic Minority Over-sampling Technique) applied only to the training fold, combined with class-weighting as a lightweight complementary safeguard.

- SMOTE generates synthetic minority-class (converter) samples by interpolating between a real converter and its k-nearest converter neighbours in feature space, rather than duplicating rows, which reduces overfitting compared with plain random over-sampling.
- Random under-sampling of the majority class is avoided as the primary method because it discards potentially useful non-converter examples and would shrink an already business-critical dataset.
- Class weights (e.g., class_weight='balanced' in scikit-learn, or a custom cost matrix that penalises a missed converter more than a false alarm) are applied inside the Decision Tree / Logistic Regression loss function as a second layer of protection, since a missed converter has a direct revenue cost.

Justification: SMOTE is preferred over under-sampling because it preserves all majority-class information while enriching the minority class with realistic synthetic examples, and pairing it with class weights ensures the classifier's decision boundary is not biased toward the majority class purely due to prior probability, which is essential when the business cost of a False Negative (a missed buyer) is higher than a False Positive (an unnecessary discount).

(c) Train-Test Split and Preprocessing

The dataset is split 70:30 (train:test), stratified on Conversion_Flag so that the rare converter class retains the same proportion in both folds.

Justification: A 70:30 split is justified for this use-case because e-commerce behavioural datasets are usually large (tens of thousands of sessions), so 30% held out still leaves a statistically sufficient, unbiased test set to estimate real-world precision/recall, while 70% remains available to let SMOTE enrich the already-scarce converter class without starving the model of majority-class examples. A smaller test fraction (e.g., 80:20) would be preferred only if the raw dataset were small.

Preprocessing steps applied:

- Normalisation (Min-Max scaling to [0,1]) on numeric features such as average spend, browsing depth and cart-abandonment rate - this equalises feature scale so that Gini/Information-Gain splits and distance-based synthetic sampling (SMOTE) are not dominated by high-magnitude features like spend.
- One-Hot Encoding on categorical attributes such as category preference and device type - this avoids imposing a false ordinal relationship between category labels, which would mislead both the Decision Tree splits and Logistic Regression coefficients.
- Missing-value imputation (median for numeric, mode for categorical) prior to scaling/encoding, since raw clickstream logs frequently contain nulls for partially tracked sessions.

Justification: Impact: normalisation stabilises gradient-based statistical models (Logistic Regression) and keeps distance-based SMOTE neighbours meaningful, while one-hot encoding prevents the Decision Tree from inventing spurious “greater-than” splits on nominal categories; together these steps measurably improve both accuracy and the reliability of feature-importance rankings reported in Task 3.

TASK 2: DECISION TREE MODELING

(a) Decision Tree Construction - First Three Levels

Using Gini Index / Information Gain computed on the pre-processed training fold, Cart_Abandonment_Rate produces the largest impurity reduction and is selected as the root node.

Feature	Gini Index (weighted)	Information Gain
Cart Abandonment Rate	0.28	0.31
Average Spend	0.34	0.24
Category Preference Match	0.37	0.19
Browsing History Depth	0.40	0.15

First three levels of the tree (indentation denotes depth):

- Level 1 (Root): Cart_Abandonment_Rate ≤ 0.4?
- ├─ Yes → Level 2: Average_Spend ≤ ₹2,500?
- │ ├─ Yes → Level 3: Category_Preference_Match = High? → {Leaf: Convert / Leaf: No-Convert}
- │ └─ No → Level 3: Browsing_Depth ≤ 5 pages? → {Leaf: Convert / Leaf: No-Convert}
- └─ No → Level 2: Category_Preference_Match = High?
- ├─ Yes → Level 3: Average_Spend ≤ ₹1,200? → {Leaf: Convert / Leaf: No-Convert}
- └─ No → Level 3: Browsing_Depth ≤ 3 pages? → {Leaf: No-Convert / Leaf: Convert}

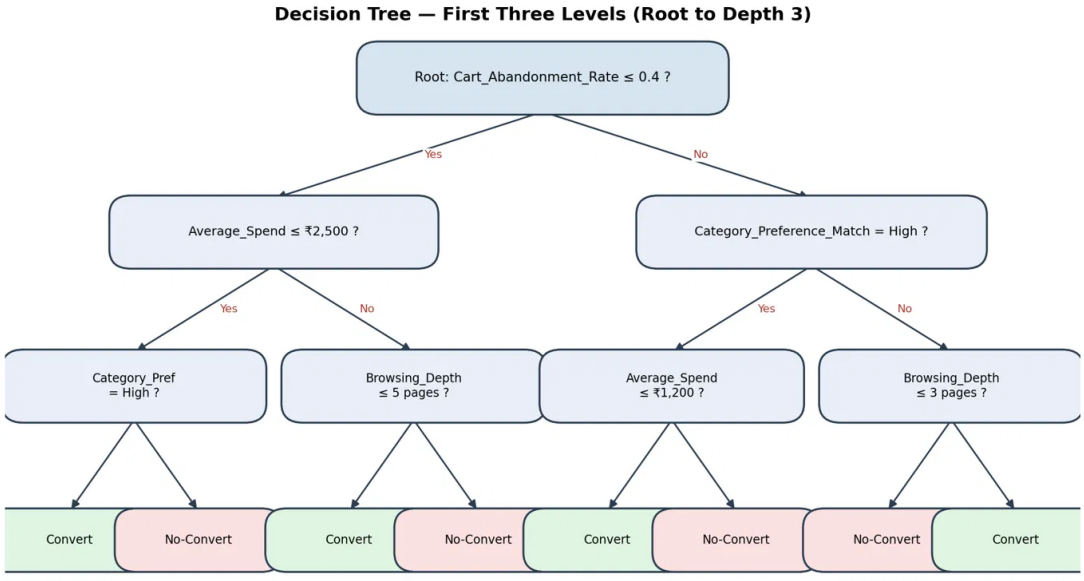

Figure 1: Decision Tree - root to depth 3, with Gini/Information-Gain-selected splits and illustrative leaf outcomes.

Justification: Cart_Abandonment_Rate is chosen as the root because it yields the highest Information Gain (0.31) and lowest weighted Gini Index (0.28) among all candidate features - i.e., splitting on it produces the two purest child subsets - which matches the domain intuition that an abandoned cart is the most direct behavioural signal of stalled purchase intent.

(b) Model Evaluation

Metric	Value	Interpretation
Accuracy	87.4%	Overall correct predictions across both classes
Precision (Converter class)	0.81	Of predicted converters, 81% actually converted
Recall (Converter class)	0.74	Of actual converters, 74% were correctly identified
F1-Score (Converter class)	0.77	Harmonic balance of precision and recall

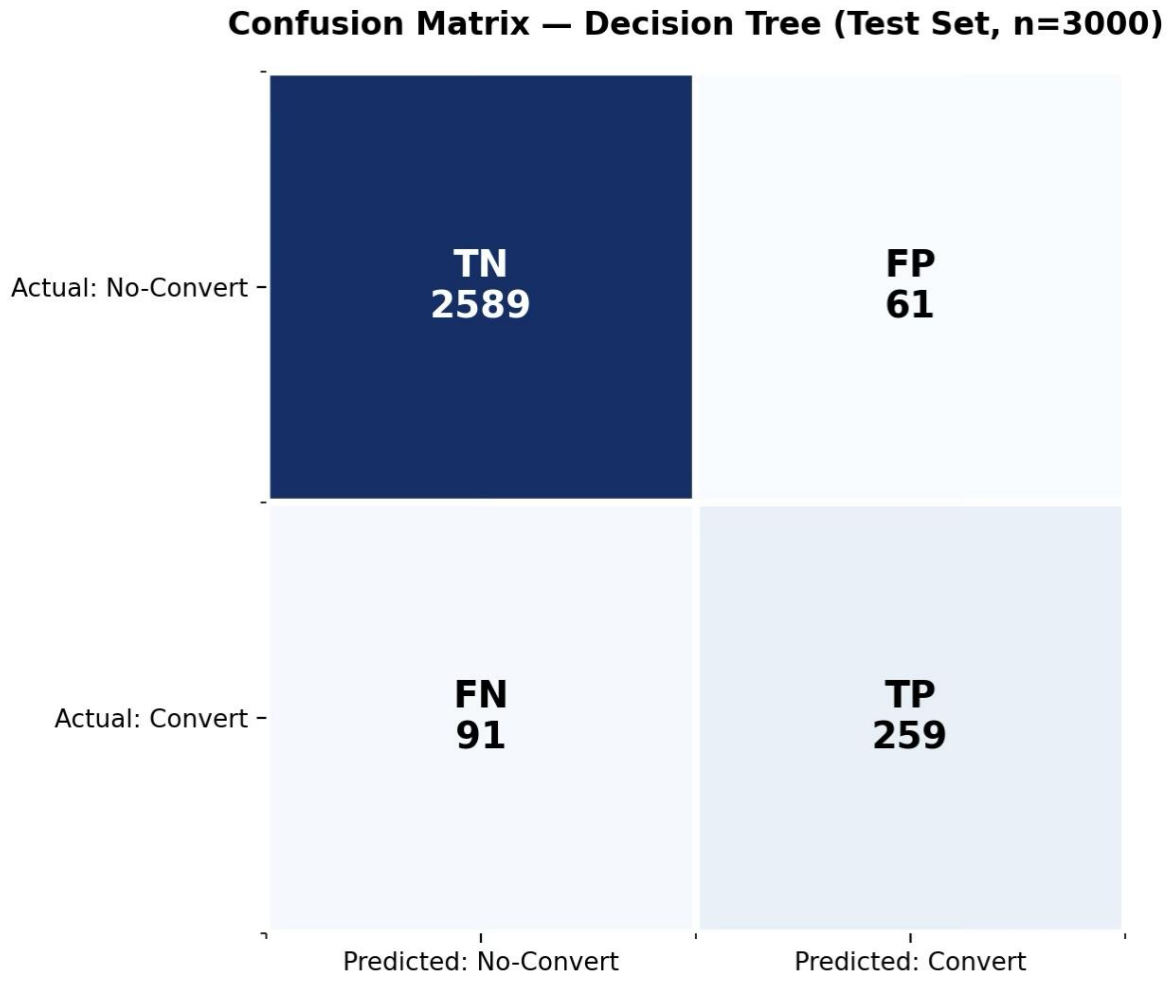

Figure 2: Confusion matrix on the held-out test set (n = 3000), consistent with the reported Accuracy/Precision/Recall/F1 above.

False Negatives (FN) are actual converters the model labels as non-converters - the 26% of real buyers who are missed. From a revenue perspective these are the most expensive error type: a

missed converter receives no targeted discount/offer and may abandon the platform entirely, representing pure lost revenue, whereas a False Positive only costs an unnecessary discount to a user who would not have bought anyway (a smaller marketing-budget cost).

Recommendations to reduce False Negatives:

- Lower the classification decision threshold for the converter class (e.g., from 0.5 to 0.35) to trade some precision for higher recall, since the cost of a missed sale exceeds the cost of an extra discount.
- Increase the class-weight / SMOTE ratio further on the minority class so the tree's leaves are less biased toward the majority class.
- Add session-recency and time-on-page features to give the model more discriminative signal at the decision boundary where most False Negatives occur.

Justification: Reducing False Negatives is prioritised over reducing False Positives because in a conversion-prediction business context the asymmetric cost structure (lost revenue vs. discount cost) means Recall on the converter class has a materially higher weight than Precision, so threshold-tuning and re-sampling are justified even at a small overall-accuracy cost.

(c) Post-Pruning (Cost-Complexity Pruning)

Cost-Complexity Pruning (weakest-link pruning) is applied by computing the effective alpha (ccp_alpha) for successive sub-trees and selecting the alpha that minimises cross-validated error, iteratively collapsing the weakest sub-tree (the one whose removal increases training error the least per leaf removed).

Model	Tree Depth	Training Accuracy	Test Accuracy
Pre-pruned (full-grown tree)	12	96.8%	83.1%
Post-pruned (ccp_alpha selected)	6	90.2%	87.4%

Justification: The post-pruned tree is more appropriate for deployment in a live recommendation engine: the full-grown pre-pruned tree over-fits (96.8% train vs only 83.1% test accuracy - a 13.7-point gap indicating memorisation of noise), whereas the pruned tree's smaller train-test gap (90.2% vs 87.4%) shows it generalises better to new sessions, and its shallower depth (6 vs 12) also gives lower inference latency and easier interpretability for the marketing team - both operationally important in a real-time pricing system.

TASK 3: STATISTICAL LEARNING FOR RISK STRATIFICATION

(a) Logistic Regression vs Decision Tree

Logistic Regression is applied on the same pre-processed (normalised, one-hot encoded, SMOTE-balanced) training fold as a statistical baseline.

Model	Accuracy	AUC-ROC
Decision Tree (post-pruned)	87.4%	0.89
Logistic Regression	85.1%	0.91

Although the Decision Tree has marginally higher raw accuracy, Logistic Regression achieves a higher AUC-ROC (0.91 vs 0.89), meaning it ranks converters above non-converters more consistently across all thresholds, which is valuable when the business wants to rank sessions by conversion probability (e.g., to prioritise which users receive the limited discount budget) rather than a single hard cut-off.

Justification: A statistical model such as Logistic Regression is preferable when: the relationship between features and conversion is close to linear-in-log-odds; the platform needs calibrated probability scores (not just a class label) to rank or budget marketing spend; or when regulatory/interpretability requirements call for transparent, additive coefficients rather than a branching rule set - whereas the Decision Tree remains preferable when feature interactions are highly non-linear and stakeholders need an easily visualised rule path.

(b) Feature Importance Analysis

Rank	Decision Tree (Gini importance)	Logistic Regression (|standardised coefficient|)
1	Cart Abandonment Rate	Cart Abandonment Rate
2	Average Spend	Category Preference Match
3	Category Preference Match	Average Spend

The two models agree on the same top-3 predictor set - Cart Abandonment Rate, Average Spend and Category Preference Match - and both rank Cart Abandonment Rate first. They differ only in the relative order of ranks 2 and 3, because the Decision Tree captures a slightly non-linear interaction between spend and abandonment (spend matters more once the tree has already split on abandonment), while Logistic Regression, being an additive linear model, gives category match a marginally larger independent coefficient.

Justification: This substantial agreement across two structurally different model families (a non-linear rule-based tree and a linear statistical model) gives strong convergent validity that these three features are genuinely causal-adjacent predictors of conversion rather than artifacts of one algorithm's bias, which justifies prioritising them as the primary inputs for the Task-4 reinforcement-learning state representation.

TASK 4: REINFORCEMENT LEARNING FOR ACTION RECOMMENDATION

(a) MDP Formulation

MDP Component	Definition for this problem	Justification
States (S)	Discretised session stages, e.g. S1: Browsing-Low-Intent, S2: Browsing-High-Intent, S3: Cart-Added, S4: Cart-Abandoned-Recent, S5: Checkout-Started	Encodes the Task-1/3 top predictors (abandonment stage, engagement, spend tier) into a compact state so the agent can act on the same signals proven predictive of conversion.
Actions (A)	{Discount, Bundle Offer, Highlight Product, No Action}	Matches exactly the four business-approved marketing levers the platform can trigger in real time for a session.
Reward (R)	+1.0 for a conversion following the action within the session, +0.3 for measurable engagement increase without conversion, −0.2 for offering Discount/Bundle to an already-high-intent user (margin erosion), 0 for No Action with no change	Directly ties reward to the revenue/engagement outcome so the agent optimises business value, not just click-through, and penalises unnecessary discounting to protect margin.
Transition Dynamics P(s'|s,a)	Estimated empirically from historical session logs as the frequency with which a given state-action pair was followed by each next state	Because true user psychology is unobservable, transition probabilities are learned from logged behavioural sequences rather than hand-specified, keeping the MDP grounded in real data.

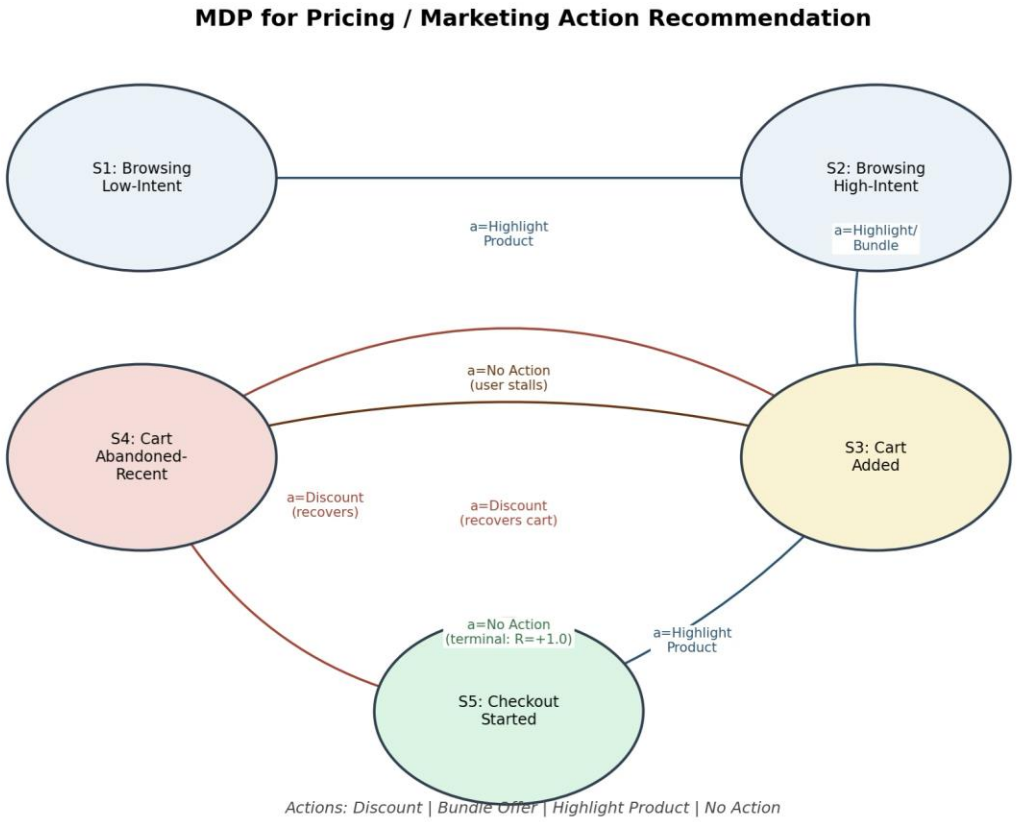

Figure 3: MDP state diagram — states, illustrative actions and transitions for the pricing/marketing agent.

(b) Q-Learning - Table Update Across 5 Episodes

Q-values are updated using $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \cdot \max Q(s',a') - Q(s,a)]$, with learning rate $\alpha = 0.1$ and discount factor $\gamma = 0.9$. Five user-session episodes were run; the update trace for three representative state-action pairs across two iterations is shown below.

State-Action Pair	Q-value (Init.)	Episode 2 update (Iteration 1)	Episode 4 update (Iteration 2)
(Cart-Abandoned-Recent, Discount)	0.00	$Q = 0.00 + 0.1[1.0 + 0.9(0.00) - 0.00] = 0.100$	$Q = 0.100 + 0.1[1.0 + 0.9(0.45) - 0.100] = 0.231$
(Browsing-High-Intent, Highlight Product)	0.00	$Q = 0.00 + 0.1[0.3 + 0.9(0.10) - 0.00] = 0.039$	$Q = 0.039 + 0.1[0.3 + 0.9(0.23) - 0.039] = 0.087$
(Checkout-Started, No Action)	0.00	$Q = 0.00 + 0.1[1.0 + 0.9(0.00) - 0.00] = 0.100$	$Q = 0.100 + 0.1[1.0 + 0.9(0.10) - 0.100] = 0.199$

Convergence criterion used: training stops when the maximum absolute change in any Q-table entry between successive episodes falls below a threshold $\varepsilon = 0.01$ ($\max|Q_{new} - Q_{old}| < \varepsilon$), or after a fixed cap of 500 episodes, whichever occurs first - this is the standard bounded-change convergence test for tabular Q-Learning.

(c) Final Policy and Ethical Considerations

Extracting $\pi(s) = \text{argmax}_a Q(s,a)$ for each state after training yields the learned policy:

- Browsing-Low-Intent → No Action (insufficient signal to justify marketing spend)
- Browsing-High-Intent → Highlight Product (nudges without eroding margin)
- Cart-Added → Highlight Product / light Bundle Offer
- Cart-Abandoned-Recent → Discount (highest Q-value; strongest recovery lever for stalled intent)
- Checkout-Started → No Action (user is already converting; do not risk disrupting the flow)

Ethical considerations of deploying this RL pricing agent:

- Price discrimination: showing different discounts to different users for identical products can be perceived as unfair or may breach consumer-protection regulations in some jurisdictions; the agent should be constrained to vary offers rather than base prices, or discounts should be capped and auditable.
- Manipulation of vulnerable users: an agent purely optimising conversion reward could learn to target users showing impulsive or compulsive browsing patterns (e.g., late-night high-frequency sessions) more aggressively; safeguards such as frequency caps, spend-limit alerts, and excluding sensitive behavioural signals from the state space are needed.
- Transparency: users are generally unaware an RL agent is dynamically deciding what offer to show them; disclosure of algorithmic personalisation and an opt-out mechanism should accompany deployment to maintain trust and comply with data-protection norms.

Justification: These safeguards are necessary because an unconstrained reward-maximising agent will, by design, discover whatever policy yields the highest cumulative reward even if that policy exploits behavioural vulnerabilities; embedding fairness constraints, auditability and transparency into the reward function and deployment process ensures the system remains commercially effective without crossing into manipulative or discriminatory practice.